

ABC457 D - Raise Minimum 题解

题意概括

给定一个长度为 N 的序列 A 和整数 K 。

每次操作可以选择一个下标 i ，把 A_i 增加 i 。总操作次数最多为 K 次。

要求在不超过 K 次操作的前提下，最大化操作后整个序列的最小值。

解题思路

这题的关键不是直接想“怎么操作最好”，而是先判断：

- 如果我们希望最终所有数都至少达到某个值 X
- 那么最少需要多少次操作？

如果这个最少操作次数不超过 K ，说明 X 可以做到；否则做不到。

这说明答案具有单调性：

- 较小的 X 如果可以做到
- 那么更小的值一定也可以做到
- 较大的 X 如果做不到
- 那么更大的值一定也做不到

于是可以二分答案。

1. 如何判断某个 X 是否可行

考虑第 i 个位置。

如果当前 $A_i \geq X$ ，那么这个位置不需要操作。

如果 $A_i < X$ ，那么我们需要不断对第 i 个位置执行操作。由于一次操作会让它增加 i ，所以最少需要的操作次数是：

$$\left\lceil \frac{X - A_i}{i} \right\rceil$$

也可以写成整除形式：

$$\left\lfloor \frac{X - A_i + i - 1}{i} \right\rfloor$$

把所有位置需要的最少操作次数加起来，记为 $need$ ：

- 如果 $need \leq K$ ，那么 X 可行
- 如果 $need > K$ ，那么 X 不可行

注意这里求的是“最少需要多少次”，因为题目允许操作次数小于 K ，所以只要最少次数不超过 K ，我们就可以做到。

2. 为什么可以二分

设 $check(X)$ 表示“能否让所有数都至少变成 X ”。

如果 $check(X)$ 为真，说明我们可以把所有位置都提升到至少 X 。那么目标改成更小的值时，只会更容易满足，所以也一定为真。

如果 $check(X)$ 为假，说明连 X 都做不到，那么比 X 更大的值需要的操作只会更多，也一定做不到。

因此 $check(X)$ 是一个先真后假的单调函数，可以直接二分最大可行值。

3. 二分边界如何取

- 下界可以取当前序列最小值，因为不做任何操作时就已经能达到它
- 上界可以取 $A_1 + K$

为什么上界是 $A_1 + K$ ？

因为下标 1 的位置每次操作只能增加 1，即使把全部 K 次操作都给它， A_1 最多也只能变成 $A_1 + K$ 。而整个序列的最小值不可能超过任意一个位置的最终值，所以答案一定不超过 $A_1 + K$ 。

正确性说明

下面证明上述算法一定正确。

1. 对于固定目标值 X ，第 i 个位置若想满足最终值至少为 X ，最少需要的操作次数一定是

$$\left\lceil \frac{X - A_i}{i} \right\rceil \quad (\text{当 } A_i < X \text{ 时})。$$

原因是：每次对位置 i 操作，只会让它增加 i 。少于这个次数时，累计增加量严格小于 $X - A_i$ ，无法达到 X ；达到这个次数后，就一定能让它不小于 X 。

2. 把所有位置达到 X 所需的最少操作次数相加，得到 $need$ 。

如果 $need \leq K$ ，那么按这个最少方案执行即可让所有位置都至少为 X ，因此 X 可行。

如果 $need > K$ ，由于每个位置都已经取了各自的最少需要次数，总操作数仍然超过 K ，那么任何方案都不可能在 K 次内完成，因此 X 不可行。

3. 可行性关于 X 具有单调性。

当目标值变大时，每个位置需要的最少操作次数不会减少，所以总需求也不会减少。于是存在一个最

大的可行值，二分查找恰好能找到它。

综上，算法求出的结果就是操作后序列最小值的最大可能值，算法正确。

复杂度

- 时间复杂度： $O(N \log V)$ ，其中 V 是答案的取值范围，这里二分的上界可以取到 $A_1 + K$
- 空间复杂度： $O(N)$

参考实现

```
#include<bits/stdc++.h>
using namespace std;

const int MAXN = 200000 + 5;

long long a[MAXN];

// 判断是否能让所有位置的值都至少达到 target
bool check(long long target, int n, long long k){
    __int128 need = 0;

    for(int i=1; i≤n; i++){
        if(a[i] < target){
            // 当前位置还差多少，以及最少需要多少次操作
            __int128 gap = (__int128)target - a[i];
            need += (gap + i - 1) / i;

            // 提前剪枝，避免继续累加
            if(need > k){
                return false;
            }
        }
    }

    return need ≤ (__int128)k;
}

int main(){
```

```
ios::sync_with_stdio(false);
cin.tie(nullptr);

// 读取序列长度和最多操作次数
int n;
long long k;
cin >> n >> k;

// 读入原始序列，并顺便求当前最小值
long long mn = LLONG_MAX;
for(int i=1; i≤n; i++){
    cin >> a[i];
    mn = min(mn, a[i]);
}

// 二分答案：下界是当前最小值，上界是  $a[1] + k$ 
long long left = mn;
long long right = a[1] + k;

while(left < right){
    long long mid = left + (right - left + 1) / 2;

    if(check(mid, n, k)){
        left = mid;
    } else {
        right = mid - 1;
    }
}

// 输出最大可行的最小值
cout << left;

return 0;
}
```